

JGames2D

um motor de jogos 2D em Java puro

manual ilustrado

sprites - camadas - colisão - som - tempo



Silvano Malfatti

todas as figuras foram desenhadas pelo próprio motor

O JGames2D é um motor de jogos 2D escrito em Java puro, sobre AWT e Swing. Não há nada para instalar: nenhum Maven, nenhum Gradle, nenhuma biblioteca de terceiros no caminho do desenho. O que existe são classes que você lê de cabo a rabo numa tarde.

JGames2D/

O motor. Janela, laço, sprites, camadas, texto, som, tempo e entrada. Escrito em inglês, com um bloco de comentário acima de cada método.

o seu jogo

As cenas e a classe principal. Você cria o motor, configura a janela, registra as cenas e manda rodar. O resto é por sua conta.

O menor jogo possível

```
JGEngine engine = new JGEngine();

engine.windowManager.setResolution(800, 600, 32);
engine.windowManager.setWindowTitle("meu jogo");

engine.addLevel(new MinhaCena()); // a cena de indice 0
engine.start();                  // daqui nao volta
```

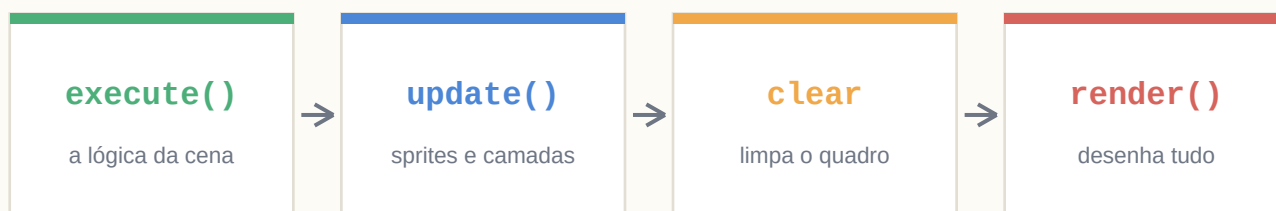
E uma cena é uma classe que estende JGLevel e responde a duas perguntas: o que carregar quando ela começa, e o que fazer a cada quadro.

```
public class MinhaCena extends JGLevel
{
    public void init()      { /* carrega o que precisa */ }
    public void execute()   { /* um quadro de logica   */ }
}
```

Sem sistema de build, de propósito

Abra a pasta como projeto Java, ponha os jars de Libs/ no classpath e rode. A raiz do projeto é a pasta de código, e é isso que faz Images/ e Sounds/ aparecerem como recursos.

Quando você chama `start()`, o motor abre uma thread própria e passa a repetir sempre a mesma sequência. Cada volta é um quadro. O alvo é um quadro a cada 33 milissegundos, mais ou menos trinta por segundo, e o tempo que o seu código gastou é descontado da pausa.



e então troca os buffers e dorme o que sobrou dos 33 ms

A regra que mais custa caro quando esquecida

Desenhar só vale dentro de `render()`. O que você desenhar em `execute()` será apagado pelo `clear` que vem logo depois, no mesmo quadro. O resultado é um HUD invisível e nenhuma mensagem de erro.

Trocar de cena

As cenas são numeradas na ordem em que você as registrou com `addLevel()`. Trocar é dizer o número, de dentro de `execute()`, e voltar em seguida: o quadro atual é abandonado e a cena nova desenha no próximo.

```
if (gameManager.inputManager.keyTyped(KeyEvent.VK_ENTER))
{
    gameManager.setCurrentLevel(2);
    return;                // nada depois disso
}
```

Uma exceção que escape da sua cena não mata a thread em silêncio: o motor registra o erro e desliga por inteiro, em vez de deixar uma janela órfã segurando a JVM.

Você nunca desenha direto na tela. Existe uma imagem do tamanho da sua resolução, o buffer de trás, e é nela que tudo acontece. Quando o quadro termina, ele é copiado para um segundo buffer, o da frente, sob trava. A janela só pinta esse segundo.

buffer de trás

Onde a sua cena desenha. É limpo e redesenhado a cada quadro. engine.graphics aponta para ele.



buffer da frente

A cópia pronta. É o único que a janela pinta, então ela nunca mostra um quadro pela metade.

Sem essa cópia, o preço aparece

Pintar direto do buffer que está sendo limpo faz a tela rasgar e piscar preto no meio do desenho. Foi exatamente o que aconteceu aqui, antes da separação.

Medidas: pergunte ao motor, não à janela

```
int largura = gameManager.windowManager.getResolutionWidth();
int altura  = gameManager.windowManager.getResolutionHeight();

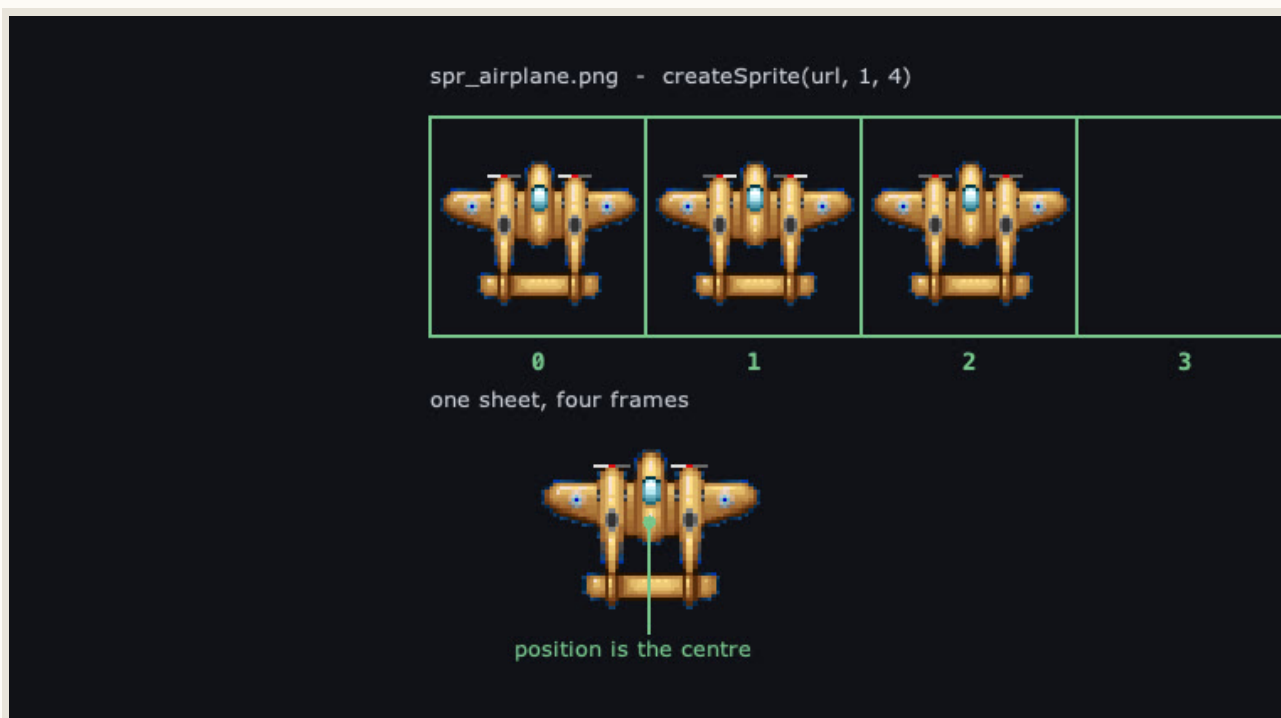
// getWidth() e getHeight(), herdados de JFrame, são outra coisa:
// medem a janela inteira, bordas e barra de título incluídas,
// e devolvem zero antes de a janela aparecer
```

Tela cheia

Chame `setFullScreen(true)` antes de mostrar a janela. No macOS o motor usa a tela cheia nativa, a mesma do botão verde, porque o modo exclusivo do Java pinta preto nas versões recentes; nos demais sistemas usa uma janela sem bordas sobre a área útil. O buffer de trás continua na resolução que você pediu e é ampliado na hora de pintar, com barras pretas para não distorcer. O ponteiro do sistema some.

Um sprite é uma imagem cortada numa grade de linhas por colunas. Você diz a grade na hora de criar, e o motor cuida do resto.

```
JGSprite aviao = createSprite(getURL("/Images/spr_airplane.png"), 1, 4);  
  
aviao.position.setXY(400, 300); // o centro, nao o canto  
aviao.zoom.setXY(2.0, 2.0); // o dobro do tamanho  
aviao.speed.setXY(0, -120); // move-se sozinho, por segundo
```



a folha, a grade de quadros e o sprite desenhado

position é o centro

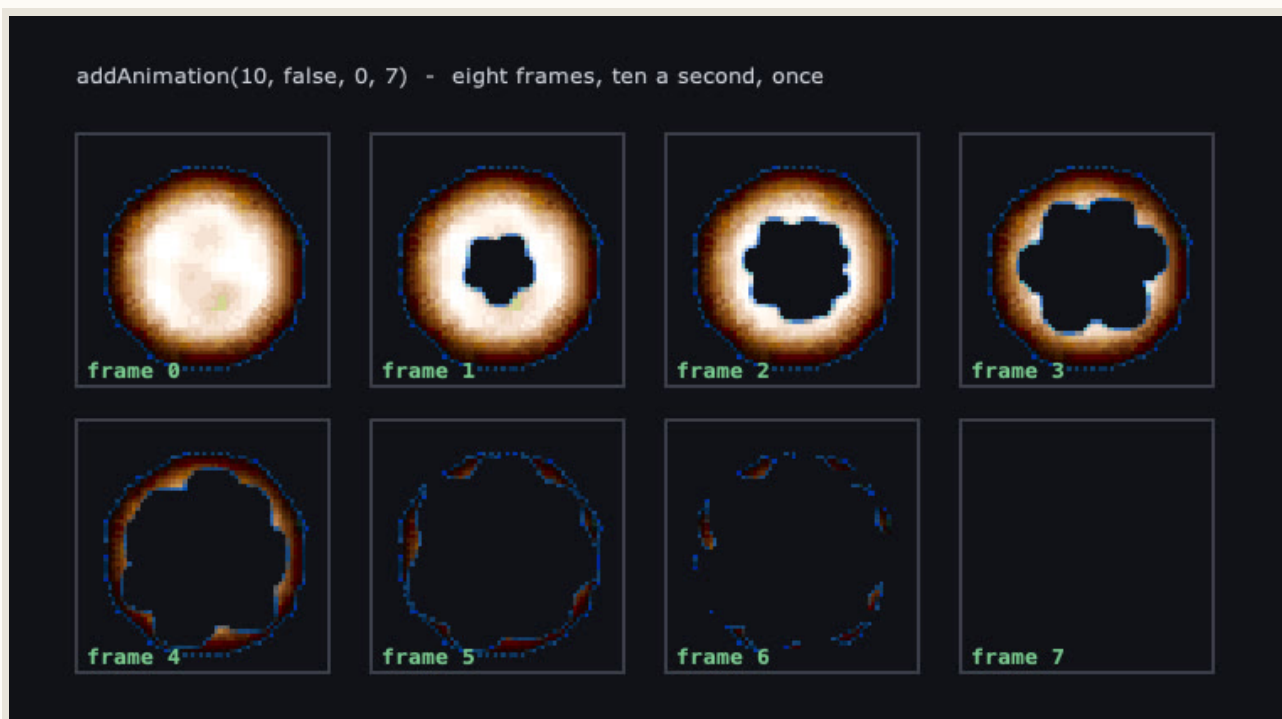
Ao desenhar, o motor recua metade do quadro em cada eixo. Isso poupa contas ao dar zoom e ao perguntar se dois sprites se encostaram.

Uma animação é uma lista de quadros da folha, com uma taxa e um aviso de repetir ou não. Um sprite pode ter várias, escolhidas pelo índice.

```
aviao.addAnimation(15, true, 0, 3);    // 0 a 3, quinze por segundo, em laço
aviao.addAnimation(10, false, 4, 7);   // 4 a 7, uma vez so

aviao.setCurrentAnimation(1);           // troca para a segunda

if (aviao.getCurrentAnimation().isEnded()) { /* acabou */ }
```



os oito quadros de uma explosão, na ordem em que aparecem

Uma animação em laço nunca termina, e `isEnded()` devolve falso para sempre nela. O relógio de cada animação desconta o intervalo em vez de zerar, então ela não atrasa aos poucos quando um quadro demora mais que o outro.

Uma camada é um mapa de blocos. O mais interessante é como ele nasce: você pinta o mapa numa imagem, um pixel por bloco, e diz qual cor vira qual quadro do conjunto de tiles.

```
JGColorIndex[] cores = new JGColorIndex[3];
cores[0] = new JGColorIndex(0, new Color(0, 0, 0)); // preto -> tile 0
cores[1] = new JGColorIndex(2, new Color(0, 255, 0)); // verde -> tile 2
cores[2] = new JGColorIndex(5, new Color(255, 255, 0)); // amarelo -> tile 5

JGOrthoLayer camada = createOrthoLayer(
    getURL("/Images/spr_elements.png"), // a folha de tiles
    getURL("/Images/lay_level.bmp"), // o mapa pintado
    cores, new JGVector2D(32, 32), true);

camada.setSpeed(new JGVector2D(0, 30)); // ela rola sozinha
```



JGOrthoLayer: a grade alinhada com a tela

O mapa se repete para sempre

Nas três projeções, chegar ao fim do mapa é voltar ao começo. Um cenário de trinta e dois blocos rola sem emenda a noite inteira.

Trocar a projeção é trocar a classe. O mapa, as cores e os tiles continuam os mesmos, e uma cena pode misturar camadas de tipos diferentes.



JGIsolayer: cada bloco é um losango, do fundo para a frente



JGTopDownLayer: visto de cima, com altura e perspectiva

Na isométrica o desenho segue a soma coluna mais linha, que cresce junto com o y da tela: é a ordem do pintor, e por isso um bloco alto cobre o de trás. Na vista de cima, um mapa de alturas diz quantos blocos há empilhados em cada célula, e as paredes se abrem para fora do ponto que a câmera olha.

A colisão entre sprites é a interseção de dois retângulos. Não há máscara por pixel nem forma poligonal: é barata e responde na hora.

```
if (aviao.collide(inimigo))
{
    inimigo.visible = false;
    explosao.setCurrentAnimation(0);
}
```



à esquerda não se tocam; à direita, sim

Contra o cenário

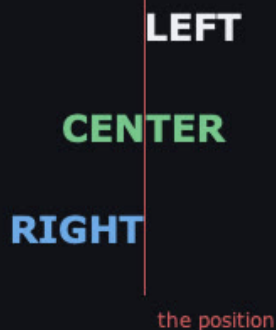
Para o mapa a pergunta é outra: `isBlockAt(x, y)` diz se há bloco sob um ponto da tela, e `getFrameIndexAt(x, y)` diz qual tile está ali, devolvendo -1 onde o mapa tem buraco. Com isso você testa o chão sob os pés do personagem sem varrer camada nenhuma.

JGFont guarda a fonte, o tamanho, a cor, o alinhamento e a posição de uma linha. A cena cria pelo nível, e o nível desenha os textos por último, depois das camadas e dos sprites: um texto nunca fica coberto.

```
JGFont pontos = createFont("verdana", Font.BOLD, 24);

pontos.text = "1500";
pontos.color = Color.white;
pontos.alignment = JGFont.RIGHT;
pontos.setPosition(largura - 20, 40); // y é a linha de base

pontos.getWidth(); // quanto ocupa, para encostar algo ao lado
```

Um diagrama sobre um fundo escuro. Uma linha vertical vermelha representa o ponto de referência. À esquerda da linha, o texto "LEFT" está em branco. À direita da linha, o texto "CENTER" está em verde e o texto "RIGHT" está em azul. Abaixo da linha, o texto "the position" está em vermelho.

LEFT

CENTER

RIGHT

the position

JGFont - a line of text as an object, drawn over everything else

os três alinhamentos, todos contra o mesmo ponto

Meça pelo objeto, não na mão

getLeft() e getRight() dizem onde as letras começam e terminam de verdade. Foi o que permitiu ao menu do MakeMeDev encostar as tags nas palavras sem medir texto nenhum.

keyPressed(tecla)

Está pressionada agora. Use para andar, mirar, acelerar: tudo que continua acontecendo enquanto o dedo estiver lá.

keyTyped(tecla)

Foi solta durante este quadro. Use para escolher, confirmar, pular: tudo que acontece uma vez por toque.

```
JGInputManager entrada = gameManager.inputManager;  
  
if (entrada.keyPressed(KeyEvent.VK_LEFT)) nave.position.setX(x - 4);  
if (entrada.keyTyped(KeyEvent.VK_SPACE)) atira();  
  
if (entrada.mouseClicked()) { /* botao solto neste quadro */ }  
JGVector2D onde = entrada.getMousePosition();
```

O evento não se gasta ao ser lido

O motor limpa os eventos de borda uma vez por quadro, no fim dele. Assim todos os objetos da cena veem o mesmo clique, e um toque que começa e termina entre dois quadros ainda é entregue.

As coordenadas do mouse já chegam no sistema do seu jogo: o motor desconta a borda da janela e desfaz a ampliação da tela cheia antes de entregar.

Há três caminhos, e escolher o certo evita quase todo problema de áudio.

JGSoundEffect

WAV curto, seis cópias abertas. Disparos seguidos se sobrepõem em vez de se cortarem, e as cópias são entregues em rodízio, porque a que acabou de tocar ainda está se esvaziando na placa.

loadTrack(url)

O mesmo objeto com uma cópia só, para música. O loop de um Clip é exato em amostras: não há buraco na emenda.

JGMusic

MP3 e OGG por streaming, pela pilha JavaZOOM. Ocupa pouca memória, mas reabre o fluxo para recomençar, então deixa um furo a cada volta.

```
JGSoundManager.init(); // uma vez, no começo

JGSoundEffect tiro =
    JGSoundManager.loadSoundEffect(getURL("/Sounds/SHOT.wav"));
tiro.setVolume(80); // 0 a 100; 100 é o arquivo
tiro.play();

JGSoundEffect trilha =
    JGSoundManager.loadTrack(getURL("/Sounds/tema.wav"));
trilha.loop(); // ate alguem mandar parar
```

O volume só desce

A escala vai de 0 a 100 e vira decibéis. Em 100 você ouve o arquivo como ele é; não há ganho acima disso. Som baixo demais se conserta no arquivo, não no código.

JGTimeManager mede quanto durou o quadro. JGTimer conta para trás a partir de um intervalo, e serve para cadência de tiro, tempo de invencibilidade, espera entre inimigos.

```
private JGTimer recarga = new JGTimer(250); // 250 ms entre tiros

public void execute()
{
    recarga.update(); // voce e quem alimenta

    if (recarga.isTimeEnded() && atirando)
    {
        atira();
        recarga.restart(250);
    }
}
```

Um timer parado é um timer que você esqueceu de atualizar

Nada os alimenta automaticamente. Chame update() no execute() da cena, sempre.

O salto depois de uma pausa

Se a janela ficar minimizada, ou você parar num ponto de interrupção, o relógio acumularia vários segundos e todos os temporizadores disparariam de uma vez ao voltar. Por isso o intervalo de um quadro é limitado a 250 ms: o jogo perde o tempo parado em vez de correr atrás dele.

As imagens e os sons passam por gerentes estáticos que guardam o que já foi lido. Pedir duas vezes o mesmo arquivo devolve o mesmo objeto.

Contagem de referências, nas imagens

Cada `loadImage()` soma um; cada `free()` subtrai. Os pixels só são descartados quando o último dono larga. É o que permite dois sprites compartilharem a mesma folha sem que a saída de um apague o desenho do outro.

```
JGImage folha = JGImageManager.loadImage(getURL("/Images/tiles.png"));  
// ... use ...  
JGImageManager.free(folha);
```

Na carga, duas coisas acontecem sozinhas

A imagem é convertida para o formato da tela, o que evita uma conversão a cada desenho, e o motor olha os pixels para descobrir a transparência que ela realmente usa. Um PNG com canal alfa mas sem nenhum pixel meio transparente vira máscara de um bit, que desenha bem mais rápido.

Todo mundo tem `free()`

A convenção do motor é que cada classe libera o que segurava e anula suas referências. Uma classe nova sua deve seguir a mesma regra, e quem cria deve chamar na hora de desmontar.

Um arquivo que não existe reclama na hora de carregar, com uma exceção clara, em vez de virar um `NullPointerException` três telas depois.

Compilar e rodar

```
javac -encoding UTF-8 -cp "Libs/*" -d out JGames2D/*.java *.java
java -cp "out:..:Libs/*" GamePrincipal
```

A raiz do projeto precisa estar no classpath: é o que faz Images/ e Sounds/ serem encontrados como recursos. Todos os arquivos são UTF-8, e o -encoding evita que a compilação dependa da configuração da máquina.

Um jar que roda em qualquer lugar

```
Tools/build-jar.sh GamePrincipal
```

O script junta motor, jogo, arte, som e os jars da JavaZOOM num arquivo só. Ele também gera um lançador .command para macOS: um jar recebido por download chega em quarentena e o sistema o recusa com um aviso enganoso de arquivo danificado, sem saída pela interface.

Por onde começar a ler

JGEngine	o laço e a lista de cenas
JGLevel	o que uma cena é capaz de criar
JGSprite	quadros, animação, movimento, colisão
JGLayer	o mapa e o que as três projeções compartilham

boa sorte, e divirta-se

o motor cabe na cabeça: leia antes de contornar